

Chapter 20 - Debugging

A bug is something that goes wrong in a program (or in our case game). Debugging is the art of fixing these bugs so that they don't occur anymore. As a programmer it's only natural to encounter lots of bugs, so debugging is a very valuable skill to have.

I wrote a short game.

function love.load()

 circle = {x = 100, y = 100}

 bullets = {}

end

function love.update(dt)

for i,v **in** ipairs(bullets) **do**

 v.x = v.x + 400 * dt

end

end

function love.draw()

 love.graphics.circle("fill", circle.x, circle.y, 50)

for i,v **in** ipairs(bullets) **do**

 love.graphics.circle("fill", v.x, v.y, 10)

end

end

function love.keypressed()

if key == "space" **then**

 shoot()

end

end

function shoot()

```
table.insert(bullets, {circle.x, circle.y})
```

end

When you press space it will shoot a bullet. Or at least that's what is supposed to happen, but it doesn't work. I don't see any bullets appearing. Let's try to find out why that is.

Here are some of the reasons I can think of for why it's not working.

- It doesn't shoot the bullets.
- It's shooting the bullets, but it's not drawing them correctly.
- It's drawing the bullets, but they are in the wrong position.

To figure out where it goes wrong we can use print. For example, let's use print inside the for-loop in love.update to check the x-position of the circle, and if it even gets there to begin with. Because if the bullets aren't spawning, the print will never be reached.

function love.update(dt)

```
for i,v in ipairs(bullets) do
```

```
    v.x = v.x + 400 * dt
```

```
    print(v.x)
```

```
end
```

end

Remember that you can add the following code at the top of your main.lua to have the output appear right away, and you don't need to close your game for it.

```
io.stdout:setvbuf("no")
```

Run the game and press space a few times to shoot. As you can see there are no prints in your output panel. Looks like we're not filling our bullets table. Let's make sure that we actually call the method shoot by putting a print there.

function shoot()

```
table.insert(bullets, {circle.x, circle.y})
```

```
-- Did you know that print takes infinite amount of arguments?
```

```
print("How many bullets?", #bullets)
```

```
end
```

Try to shoot again and you'll see that we still don't see anything printed to your output panel. Weird, because our if-statement says if key == "space", and we're definitely pressing space. But to be sure, let's print the value of key. And who knows, maybe we spelled love.keypressed wrong and it doesn't reach that code at all.

```
function love.keypressed()
```

```
-- Adding texts like these gives context to your print.
```

```
-- This is especially useful when you have multiple prints.
```

```
print("What is the key?", key)
```

```
if key == "space" then
```

```
    shoot()
```

```
end
```

```
end
```

When you try to shoot again you'll see that now there is something being printed. Looks like the value of key is nil. How is that possible, because LOVE passes the key as first argument. But wait, we don't have a parameter called key. I forgot to add it when making the function. So let's fix that.

```
function love.keypressed(key)
```

```
    print("What is the key?", key)
```

```
if key == "space" then
```

```
    shoot()
```

```
end
```

```
end
```

Okay so now when you press space, it still doesn't shoot, but something else happens. You get an error.

Reading and fixing errors

An error occurs when the code is trying to execute something that isn't possible. For example, you can't multiply a number by a string. This will give you an error

```
print(100 * "test")
```

Error

```
main.lua:1: attempt to perform arithmetic on a string value
```

Traceback

```
main.lua:1: in main chunk  
[C]: in function 'require'  
[C]: in function 'xpcall'  
[C]: in function 'xpcall'
```

Press Ctrl+C or tap to copy this error

Another example is trying to call a function that doesn't exist.

```
doesNotExist()
```

Error

```
main.lua:1: attempt to call global 'doesNotExist' (a nil value)
```

Traceback

```
main.lua:1: in main chunk  
[C]: in function 'require'  
[C]: in function 'xpcall'  
[C]: in function 'xpcall'
```

Press Ctrl+C or tap to copy this error

In our bullet shooting game we get the following error:

Error

main.lua:11: attempt to perform arithmetic on field 'x' (a nil value)

Traceback

main.lua:11: in function 'update'
[C]: in function 'xpcall'

Press Ctrl+C or tap to copy this error

So what exactly does the error tell us? Because a mistake that a lot of beginners make is that they don't realize that the error message tells you exactly why and where the error occurs.

main.lua:10:

This tells us the error occurs on line 10 (this may be a different line for you).

attempt to perform arithmetic on field 'x' (a **nil** value)

Arithmetic means a calculation, e.g. using +, -, *, etc. It tried to calculate using the field x, but x is a nil value.

So that's weird, because we give the table an x and y value, right? Well no. We add the values to the table, but we don't assign them to a field. Let's fix that.

function shoot()

 table.insert(bullets, {x = circle.x, y = circle.y})

end

And now it finally works. We can shoot bullets!

Let's look at some new code where I create a circle class, and draw it a few times (you don't necessarily have to write along).

Object = require "classic"

Circle = Object:extend()

function Circle:new()

```
self.x = 100
self.y = 100
end

function Circle:draw(offset)
    love.graphics.circle("fill", self.x + offset, self.y, 25)
end

function love.load()
    circle = Circle()
end

function love.draw()
    circle:draw(10)
    circle:draw(70)
    circle:draw(140)
    circle:draw(210)
end
```

Upon running this I get the following error:

Error

main.lua:11: attempt to index local 'self' (a number value)

Traceback

main.lua:11: in function 'draw'
main.lua:21: in function 'draw'
[C]: in function 'xpcall'

Press Ctrl+C or tap to copy this error

"Attempt to index" means it tried to find a property of something. So in this case it tried to find the property `x` on the variable `self`. But according to the error, `self` is a number value, so it can't do that. So how did this happen? We use a colon (`:`) for our function so that it automatically has `self` as first parameter, and we call the function with a colon so that it passes `self` as first argument. But apparently somewhere something went wrong. To know where it went wrong, we can use the Traceback.

The bottom part of the error tells us the "path" it took before reaching the error. It's read from down to top. You can ignore the `xpcall` line. The next line says `main.lua:21: in function 'draw'`. Interesting, let's take a look. Ah yes, I see. On line 21 I used a dot instead of a colon (`circle.draw(140)`). I changed it to a colon and now it works!

Syntax errors

A *syntax error* occurs when the game can't start to begin with, because something is wrong with the code. It tries to "read" the code but can't understand it. For example, remember how you can't have a variable name starting with a number? When you try to do so it will give you an error:

Error

Syntax error: main.lua:1: malformed number near '23test'

Take a look at the following code (again, no need to type along)

function love.load()

 timer = 0

 show = **true**

end

function love.update(dt)

 show = **false**

 timer = timer + dt

if timer > 1 **then**

```

    if love.keyboard.isDown("space") then
        show = true
    end

    if timer > 2 then
        timer = 0
    end
end

function love.draw()
    if show then
        love.graphics.rectangle("fill", 100, 100, 100, 100)
    end
end

```

It gives me the following error:

Error

Syntax error: main.lua:23: 'end' expected (to close 'function' at line 6) near '<eof>'

Traceback

```

[C]: at 0x7ffc9b103340
[C]: in function 'require'
[C]: in function 'xpcall'
[C]: in function 'xpcall'

```

<eof> means *end of file*. It expects an end at the end of the file. Is that how we fix it, placing an end at the end of the file? Well no. Somewhere I messed up, and we need to fix it correctly. It says that it expects the end to close the function at line 6, so let's start from there and go down. After opening the function I start an if-statement, and then another if-statement. I close the second if-statement and start another if-statement. I close that if-statement as well and then close the outer if-statement. No, wait, that's not right. I should

be closing the function at that point. I forgot to add an end somewhere in the function. Let me fix that.

```
function love.update(dt)

    show = false

    timer = timer + dt


    if timer > 1 then

        if love.keyboard.isDown("space") then

            show = true

        end

        if timer > 2 then

            timer = 0

        end

    end

end
```

And now it works. This is why indentation in your code is so important. It helps you see where you made a mistake like this one.

Another common mistake is one like the following:

```
function love.load()

    t = {}

    table.insert(t, {x = 100, y = 50})

end
```

Which gives me the follow error:

Error

Syntax error: main.lua:3: '}' expected near ')'

It's because I didn't close the curly brackets.

```
function love.load()

  t = {}

  table.insert(t, {x = 100, y = 50})

end
```

Asking for help

Now maybe you have a bug, and you can't fix it. You tried debugging it but you just can't figure out why it won't work, and you feel like you need help. Well then lucky for you there are plenty of people on the internet that are happy to help you. The best places to ask your question are on [forums](#) or in the [Discord](#). But asking a question is not simply asking "Hey guys I got this bug where this happens, what do I do?". You need to provide them information that they can use to help you. For example:

- What exactly goes wrong and what do you expect/want to happen?
- Explain what you have tried to do so far to fix it.
- Show the code of where (you think) it goes wrong.
- Share the .love file so that other people can try it out themselves.

But before you ask for help you may want to search for your question. It just might be that it's a common question and it has been answered multiple times.

And remember, no one is obligated to help you, and the ones that do all do it for free, so be kind :)

Rubber duck debugging

You could also get a rubber duck. There's this thing called [rubber duck debugging](#). The idea is that when you explain what you're doing in your code, you often realize what you're doing wrong and fix the bug yourself. So instead of explaining it to someone, you explain it to your rubber duck. I have rubber duck myself, his name is Hendrik!



Summary

We can use print to find the source of our bug. Error message tell us exactly what is going wrong. Syntax errors are errors that occur because it can't "read" the code correctly. Indentation is useful because it prevents us from getting end of line-errors. We can ask for help on the internet but we should provide enough information on what is going on to make it less difficult on the people helping us.